

Permutation-Schedule Dominance for Two-Stage Assembly Flow Shops with Divisible Work

Eric Bowman^{1*}

¹King, Berlin, Germany.

Corresponding author(s). E-mail(s): eric.bowman@king.com;

Abstract

Permutation schedules are known to dominate in the classical two-stage assembly flow shop for makespan and weighted completion time. We extend this dominance property to a divisible-work model in which each stage-1 machine contains k identical parallel workers, job workloads may differ by machine, and work is allocated integrally across workers. For any feasible schedule, we construct a permutation schedule whose stage-2 completion time is no larger for every job. This pointwise domination implies optimality of permutation schedules for every regular stage-2 objective, including makespan, total weighted completion time, and maximum lateness. The proof is constructive and yields the synchronized order explicitly from the release times of the original schedule. The construction is based on a single capacity inequality— k workers can complete at most kt units of work by time t —combined with a water-filling allocation that balances worker loads. The classical model is recovered when $k = 1$. Small instances are exhaustively verified computationally, and the core dominance and feasibility arguments are formalized in Lean 4.

Keywords: assembly flow shop, permutation schedule, divisible work, weighted completion time, flow shop scheduling

MSC Classification: 90B35

1 Introduction

The two-stage assembly flow shop, introduced by [Lee et al. \(1993\)](#), consists of m parallel machines at stage 1, each processing all n jobs without preemption, followed

by a single assembly machine at stage 2 that can begin job j only after all stage-1 machines have completed j .

Potts et al. (1995) proved the problem is NP-hard in the strong sense for $m \geq 2$ and that *permutation schedules dominate*: the search for a $C_{\max}$ -optimal schedule may be restricted to schedules where all stage-1 machines use the same job ordering, reducing the search space from $(n!)^m$ orderings to $n!$. Tozkapan et al. (2003) extended permutation-schedule dominance to the total weighted flowtime objective. Hariri and Potts (1997) developed a branch-and-bound algorithm with dominance theorems for instances up to $n = 8000$, $m = 10$. Hadda et al. (2024) contributed new dominance rules and an exact branch-and-bound, solving instances up to $n = 1000$, $m = 8$.

Koulamas and Kyparisis (2007) studied a variant with *uniform* parallel machines at stage 1 (machines with different speeds). Their model differs from ours: uniform machines have one processor per machine at varying speeds, whereas our model has k identical workers per machine sharing divisible work.

We distinguish several adjacent lines of work. The malleable-job literature (Drozdowski, 2009) studies divisible work on parallel processors but without the assembly constraint. Al-Anzi and Allahverdi (2007) study the two-stage assembly problem with setup times, using permutation dominance to restrict the heuristic search space. Hatami et al. (2013) consider distributed assembly permutation flow shops (multiple factories, not multiple workers per machine). Zhang and Chen (2022) study assembly scheduling with parallel teams and heterogeneous worker skills, in a multi-objective setting without dominance results. Komaki et al. (2019) survey assembly flow shop models; the divisible-work model studied here does not appear in their taxonomy. None of these states the dominance result proved here.

1.1 Contribution

We prove that permutation-schedule dominance extends to a divisible-work generalization where each stage-1 machine has $k \geq 1$ identical parallel workers and work requirements $w_{i,j}$ may differ across machines. Setting $k = 1$ recovers the classical model. The specific contributions are:

1. The extension from $k = 1$ to $k \geq 1$ workers per machine with integer work division, including the case $d_{i,j,\ell} \geq 0$ (some workers idle) with $w_{i,j} \geq 1$.
2. Machine-dependent work requirements $w_{i,j}$.
3. A shorter and more direct proof than the exchange-argument technique of Potts et al. (1995). Their proof transforms a schedule by pairwise job exchanges, tracking the assembly barrier at each step; ours constructs the permutation schedule directly from a single inequality without intermediate exchanges. The construction runs in $O(n \log n + mnk)$ time.

2 Model

Definition 1 (Divisible-Work Two-Stage Assembly Flow Shop) An instance consists of:

- n jobs, indexed $j \in [n] = \{1, \dots, n\}$.
- m stage-1 machines, each with k identical workers ($k \geq 1$).

- One stage-2 machine (with any number of workers; the stage-2 schedule is inherited unchanged, so the stage-2 structure does not affect the dominance result).
- Work requirements $w_{i,j} \in \mathbb{Z}_{>0}$ for each machine $i \in [m]$ and job $j \in [n]$.
- Stage-2 work requirements $q_j \in \mathbb{Z}_{>0}$ for each job j .

Definition 2 (Schedule) A schedule $S = (\sigma, \mathbf{d}, \tau, \mathbf{e})$ consists of:

- Permutations $\sigma_i \in \mathfrak{S}_n$ for each machine i : machine i processes jobs in order $\sigma_i(1), \dots, \sigma_i(n)$.
- Work divisions $d_{i,j,\ell} \in \mathbb{Z}_{\geq 0}$ for each machine i , job j , worker $\ell \in [k]$, satisfying $\sum_{\ell} d_{i,j,\ell} = w_{i,j}$.
- A permutation $\tau \in \mathfrak{S}_n$ and work divisions $e_{j,\ell}$ for stage 2, with $\sum_{\ell} e_{j,\ell} = q_j$.

All processing is non-preemptive: each worker completes its assigned portion of a job before starting the next.

Definition 3 (Completion Times) Worker ℓ on machine i processes jobs sequentially in order σ_i . The cumulative finish time after position s is $F_{i,\ell,s} = \sum_{t=1}^s d_{i,\sigma_i(t),\ell}$. The completion time of job j on machine i is $C_{i,j} = \max_{\ell} F_{i,\ell,\sigma_i^{-1}(j)}$. All completion times are non-negative integers.

The *release time* of job j to stage 2 is $r_j = \max_{i \in [m]} C_{i,j}$.

Stage-2 worker ℓ starts job $\tau(s)$ at time $\max(r_{\tau(s)}, F_{\ell,s-1}^2)$ and finishes at $F_{\ell,s}^2 = \max(r_{\tau(s)}, F_{\ell,s-1}^2) + e_{\tau(s),\ell}$, with $F_{\ell,0}^2 = 0$. We write $C_j^{(2)} = \max_{\ell} F_{\ell,\tau^{-1}(j)}^2$ for the stage-2 completion time of job j , and $C_{\max}(S) = \max_j C_j^{(2)}$ for the makespan.

Definition 4 (Permutation Schedule) A schedule is a *permutation schedule* if $\sigma_1 = \sigma_2 = \dots = \sigma_m$.

In the three-field notation of [Graham et al. \(1979\)](#), the problem is $\text{FH2}(P_{m \times k}, P_{k'}) \mid \text{assembly, div} \mid C_{\max}$, where “div” denotes divisible integer work.

3 Main Result

Theorem 5 (Pointwise Stage-2 Domination) *For every schedule S , there exists a permutation schedule S^* with $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every job j .*

Corollary 6 *Permutation schedules are optimal for every regular (non-decreasing) function of stage-2 completion times, including $C_{\max}$, $\sum_j v_j C_j^{(2)}$ for any $v_j > 0$, and $\max_j (C_j^{(2)} - d_j)$ for any due dates d_j .*

The proof of Theorem 5 uses two lemmas.

Lemma 7 (Capacity Bound) *Let S be any schedule with release times r_j . Let π be a permutation sorting jobs by non-decreasing release time: $r_{\pi(1)} \leq \dots \leq r_{\pi(n)}$. Then for every machine i and every $s \in [n]$:*

$$W_i(s) := \sum_{t=1}^s w_{i,\pi(t)} \leq k \cdot r_{\pi(s)}.$$

Proof In schedule S , machine i completes job $\pi(t)$ by time $C_{i,\pi(t)} \leq r_{\pi(t)} \leq r_{\pi(s)}$ for each $t \leq s$. Hence by time $r_{\pi(s)}$, machine i has completed at least jobs $\{\pi(1), \dots, \pi(s)\}$, whose total work is $W_i(s)$. Since k workers each contribute at most $r_{\pi(s)}$ time units: $W_i(s) \leq k \cdot r_{\pi(s)}$. $\square$

Lemma 8 (Water-Filling Feasibility) *Given machine i processing jobs in order π , allocating work one unit at a time to the currently least-loaded worker achieves completion time $\lceil W_i(s)/k \rceil$ at each position s .*

Proof By induction on s . The invariant: after position s , worker loads lie in $\{\lfloor W_i(s)/k \rfloor, \lceil W_i(s)/k \rceil\}$ (loads differ by at most 1 and sum to $W_i(s)$).

At position s , distribute $w_{i,\pi(s)}$ units one at a time, each to a currently least-loaded worker. Since the invariant holds at $s-1$, loads before allocation differ by at most 1. Adding units one-at-a-time to the minimum maintains the spread ≤ 1 . The total becomes $W_i(s)$, so the maximum is $\lceil W_i(s)/k \rceil$.

When $w_{i,\pi(s)} < k$, some workers are idle for this job ($d_{i,\pi(s),\ell} = 0$), which the model permits. When $w_{i,\pi(s)} \geq k$, the allocation can also be arranged so that every worker receives at least one unit: give each worker one unit first (adding one unit to every load preserves the spread), then distribute the remaining $w_{i,\pi(s)} - k$ units least-loaded-first.

The division is also available in closed form: after position s , exactly $W_i(s) \bmod k$ workers hold $\lceil W_i(s)/k \rceil$ units and the rest hold $\lfloor W_i(s)/k \rfloor$, so the divisions for each job are computable in $O(k)$ time. This gives the $O(n \log n + mnk)$ construction time claimed in Section 1. $\square$

Proof of Theorem 5 Given schedule S with release times r_j , let π sort jobs by non-decreasing r_j . Construct S^* :

1. Every stage-1 machine uses permutation π .
2. Each machine i independently uses water-filling (Lemma 8) to allocate $w_{i,\pi(s)}$.
3. Stage 2 uses the same τ and $\mathbf{e}$ as S .

By Lemma 8, $C_{i,\pi(s)}^* = \lceil W_i(s)/k \rceil$. By Lemma 7, $W_i(s) \leq k \cdot r_{\pi(s)}$. Since $r_{\pi(s)}$ is a non-negative integer: $\lceil W_i(s)/k \rceil \leq r_{\pi(s)}$ for all i, s . Therefore $r_j^* \leq r_j$ for all j .

Since $r_j^* \leq r_j$ and the stage-2 schedule is unchanged, by induction on stage-2 positions: $F_{\ell,s}^{2*} \leq F_{\ell,s}^2$ for all ℓ, s . Therefore $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every j . $\square$

Remark 9 (Classical recovery) When $k = 1$, water-filling is the identity and the theorem recovers the permutation-schedule dominance of Potts et al. (1995).

Remark 10 (The permutation π) The construction uses π sorted by the release times of the given schedule S . The result is existential: for every S , there exists a permutation schedule at least as good.

Remark 11 (Stage-2 re-optimization) The construction inherits stage 2 unchanged. Re-optimizing the stage-2 work divisions or ordering can only improve the result but is not needed for dominance.

Remark 12 (Scope of the construction) Four generalizations follow from the same proof verbatim. (i) *Machine-dependent worker counts*: if machine i has k_i workers, Lemma 7 gives $W_i(s) \leq k_i r_{\pi(s)}$ and water-filling yields $\lceil W_i(s)/k_i \rceil \leq r_{\pi(s)}$; nothing else changes. (ii) *Real-valued work*: with divisible real-valued workloads the integrality step is unnecessary—equal splitting gives completion $W_i(s)/k \leq r_{\pi(s)}$ directly. (iii) *Inserted idle time*: if schedules may idle workers, Lemma 7 still holds (each worker completes at most $r_{\pi(s)}$ units of work by time $r_{\pi(s)}$), and the constructed S^* uses no idle time, so the domination is unchanged. (iv) *General downstream structure*: the proof of Theorem 5 uses only that stage-2 completion times are componentwise non-decreasing in the release-time vector, so the single assembly stage may be replaced by any downstream system with this monotonicity property.

Example.

Let $n = 3$, $m = 2$, $k = 2$. Machine 1: $(w_{1,A}, w_{1,B}, w_{1,C}) = (6, 4, 2)$; machine 2: $(w_{2,A}, w_{2,B}, w_{2,C}) = (2, 6, 4)$.

Schedule S : machine 1 uses A, B, C ; machine 2 uses B, C, A . Water-filling gives: machine 1 completes A at 3, B at 5, C at 6; machine 2 completes B at 3, C at 5, A at 6. Release times: $r_A = 6$, $r_B = 5$, $r_C = 6$.

Sort: $\pi = (B, A, C)$. Capacity bound holds: $W_1(1) = 4 \leq 10$, $W_2(1) = 6 \leq 10$, etc. Water-filling with π : $r_B^* = 3 \leq 5$, $r_A^* = 5 \leq 6$, $r_C^* = 6 \leq 6$. Pointwise domination.

4 Computational Verification and Lean 4 Formalization

We verified Theorem 5 by exhaustive enumeration across 27 homogeneous-work configurations ($w_{i,j} = w_j$; 3,846,136 schedules) and 12 heterogeneous-work configurations ($w_{i,j}$ varying across machines; 167,120 schedules), with $n \in \{2, \dots, 5\}$, $m \in \{2, 3\}$, $k \in \{2, 3\}$. In every case, a permutation schedule achieved the pointwise minimum.

Each step of the proof is formalized in Lean 4 (v4.29.0, Mathlib) with zero unresolved axioms: the capacity bound (Lemma 7) is derived from a concrete machine model with per-worker cumulative finish times; the water-filling allocation (Lemma 8) is constructed explicitly—workers take work units round-robin—and shown to achieve $\lceil W_i(s)/k \rceil$ at every position; and the release-time domination composes with stage-2 monotonicity to give the pointwise conclusion.¹ The development contains no unproven assumptions (no `sorry`). It also includes a machine-checked counterexample showing

¹Source code, verification data, and Lean formalization: <https://github.com/ebowman/synchronization-paper-artifact>.

that the bounds available to a pairwise-interchange argument cannot establish the theorem at the release-time abstraction, which motivated the direct construction.

5 Conclusion

For any schedule in the divisible-work two-stage assembly flow shop, we have constructed a permutation schedule with $C_j^{(2)}(S^*) \leq C_j^{(2)}(S)$ for every job j , using a capacity bound and water-filling. This extends the classical results of Potts et al. (1995) and Tozkapan et al. (2003) from $k = 1$ to arbitrary k , with machine-dependent work requirements.

Open questions include whether the result extends to deeper assembly structures (more than two stages), stochastic processing times, or sequence-dependent setup costs.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. The sole author conceived the work, developed the proofs and formalization, and wrote the manuscript.

Data availability. The verification data supporting the quantitative claims of Section 4 are available in the repository below.

Code availability. The Lean 4 formalization and Scala verification code are available at <https://github.com/ebowman/synchronization-paper-artifact> and archived at <https://doi.org/10.5281/zenodo.20075388> (this submission corresponds to version 1.1.0).

Generative AI disclosure. During the preparation of this work, the author used Claude for critique of exposition, structural feedback, and language revision. The author reviewed and edited the manuscript and takes full responsibility for the content.

References

- Lee, C.-Y., Cheng, T.C.E., Lin, B.M.T.: Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. *Management Science* **39**(5), 616–625 (1993) <https://doi.org/10.1287/mnsc.39.5.616>
- Potts, C.N., Sevast'janov, S.V., Strusevich, V.A., Van Wassenhove, L.N., Zwaneveld, C.M.: The two-stage assembly scheduling problem: Complexity and approximation. *Operations Research* **43**(2), 346–355 (1995) <https://doi.org/10.1287/opre.43.2.346>
- Tozkapan, A., Kırça, Ö., Chung, C.-S.: A branch and bound algorithm to minimize the total weighted flowtime for the two-stage assembly scheduling problem. *Computers & Operations Research* **30**(2), 309–320 (2003) [https://doi.org/10.1016/S0305-0548\(01\)00098-3](https://doi.org/10.1016/S0305-0548(01)00098-3)

- Hariri, A.M.A., Potts, C.N.: A branch and bound algorithm for the two-stage assembly scheduling problem. *European Journal of Operational Research* **103**(3), 547–556 (1997) [https://doi.org/10.1016/S0377-2217\(96\)00312-8](https://doi.org/10.1016/S0377-2217(96)00312-8)
- Hadda, H., Dridi, N., Hajri-Gabouj, S.: On the two-stage assembly flow shop problem. *TOP* **32**(2), 224–244 (2024) <https://doi.org/10.1007/s11750-024-00664-0>
- Koulamas, C., Kyparisis, G.J.: A note on the two-stage assembly flow shop scheduling problem with uniform parallel machines. *European Journal of Operational Research* **182**(2), 945–951 (2007) <https://doi.org/10.1016/j.ejor.2006.07.027>
- Drozdowski, M.: *Scheduling for Parallel Processing*. Springer, London (2009). <https://doi.org/10.1007/978-1-84882-310-5>
- Al-Anzi, F.S., Allahverdi, A.: A self-adaptive differential evolution heuristic for two-stage assembly scheduling problem to minimize maximum lateness with setup times. *European Journal of Operational Research* **182**(1), 80–94 (2007) <https://doi.org/10.1016/j.ejor.2006.09.011>
- Hatami, S., Ruiz, R., Andrés-Romano, C.: The distributed assembly permutation flowshop scheduling problem. *International Journal of Production Research* **51**(17), 5292–5308 (2013) <https://doi.org/10.1080/00207543.2013.807955>
- Zhang, Z., Chen, R.: Multi-objective complex product assembly scheduling problem considering parallel team and worker skills. *Journal of Manufacturing Systems* **62**, 876–893 (2022) <https://doi.org/10.1016/j.jmsy.2022.05.003>
- Komaki, G.M., Sheikh, S., Malakooti, B.: Flow shop scheduling problems with assembly operations: A review and new trends. *International Journal of Production Research* **57**(10), 2926–2955 (2019) <https://doi.org/10.1080/00207543.2018.1550269>
- Graham, R.L., Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G.: Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics* **5**, 287–326 (1979) [https://doi.org/10.1016/S0167-5060\(08\)70356-X](https://doi.org/10.1016/S0167-5060(08)70356-X)